Discrete optimization project : EURO/ROADEF 2026 Challenge

Presented by

- Kulczycki Robin
- Paulis Antoine
- Zanella Francois

Outline

- Introduction
- MIP Model
- Heuristic
- Column restriction methods
- Results
- Limitations

Introduction

Introduction – Main ideas

- ~~**Traditional Routing**~~ ➔ **Segmenting Routing** : waypoints which all traffic is forced to go through.
- **Network :** directed graph ($G = (V,A),\omega,c$).
- **Demand :** $d(s,t)$ = the traffic volume i.e. the size of the flow that needs to be routed from s to t.
- **Forwarding graph :** $FG(s,t)$ from a source s to a target t is the subgraph of G containing all arcs that belong to any shortest path.
- **Segment routing path :** succession of forwarding graphs $\langle s,\omega_1, \ldots,\omega_l,t\rangle$.

Introduction – Main ideas

- **Input :** network configuration G, a set of k demands D and, intervention and budget scenario.
- **Ouput :** a set P of k segment routing paths $\{p_d : d \in D\}$.
- **Solver :** Gurobi (Python).

MIP Model

MIP Model – Variables

- **Decision variables :**
  - <u>Binary decision variables :</u> $x_{i,j}^{dt} = \begin{cases} 1 & \text{if the segment path of demand } d \in D \text{ contains}(i,j) \text{ at time step } t \in T \\ 0 & \text{otherwise} \end{cases}$
  - <u>Load variables :</u> $\lambda(a,t) = \dfrac{\sum_{d \in D} \sum_{i,j \in V} r(i,j,a,t)\, \nu(d,t)\, x_{ij}^{d,t}}{c(a)} = \dfrac{\text{amount of flow using a}}{\text{capacity of a}}$

MIP Model – Objective

- **Objective :** minimize the load across all arcs by decreasing lexicographic order :
- **Lexicographic loop :**

For $l = 1$ to $|L|$ **do:**

$$\begin{aligned} \min \quad & z \\ \text{s.t.} \quad & \text{(a)-(e)} \\ & \lambda(a,t) \leq z, \quad \forall a \in A, \forall t \in T \\ & \lambda(a,t) \leq z^k, \forall k \in \{1, ..., l-1\} \end{aligned}$$

If the problem is unbounded or infeasible then, no solution.
Otherwise, set $z_l = z^*$
End for

- **Solution :** $z^1 \geq z^2 \geq ... \geq z^l$
- **In practice :** 5 iterations.

MIP Model - Constraints

a) **Flow conservation :**

$$\sum_{j \in V \setminus \{i\}} x_{ij}^{d,t} - \sum_{j \in V \setminus \{i\}} x_{ji}^{d,t} = \begin{cases} 1 & \text{if } i = s, \\ -1 & \text{if } i = t, \\ 0 & \text{otherwise.} \end{cases} \quad \forall i \in V,\ \forall d = (s,t) \in D,\ \forall t \in T$$

b) **Number of segments :**

$$\sum_{i,j \in V} x_{ij}^{d,t} \leq \text{maxSeg}, \quad \forall d \in D,\ \forall t \in T$$

c) **Load constraints :**

$$\sum_{d \in D} \sum_{i,j \in V} r(i,j,a,t)\, \nu(d,t)\, x_{ij}^{d,t} \leq \lambda(a,t)\, c(a), \quad \forall t \in T,\ \forall a \in A \setminus q(t)$$

d) **Budget constraints :**

$$\sum_{d \in D} \sum_{i,j \in V,\ i \neq j} \left| x_{ij}^{d,t} - x_{ij}^{d,t-1} \right| \leq \kappa(t), \quad \forall t \in T^* \xrightarrow{\text{Linearization}}$$

$$\sum_{d \in D} \sum_{i,j \in V,\ i \neq j} y_{ij}^{dt} \leq \kappa(t) \qquad \forall t \in T^*$$

$$y_{ij}^{dt} \geq x_{ij}^{dt} - x_{ij}^{d,t-1} \qquad \forall d \in D,\ \forall i,j \in V,\ i \neq j,\ \forall t \in T^*$$

$$y_{ij}^{dt} \geq -x_{ij}^{dt} + x_{ij}^{d,t-1} \qquad \forall d \in D,\ \forall i,j \in V,\ i \neq j,\ \forall t \in T^*$$

e) **Subtour Elimination constraint :**

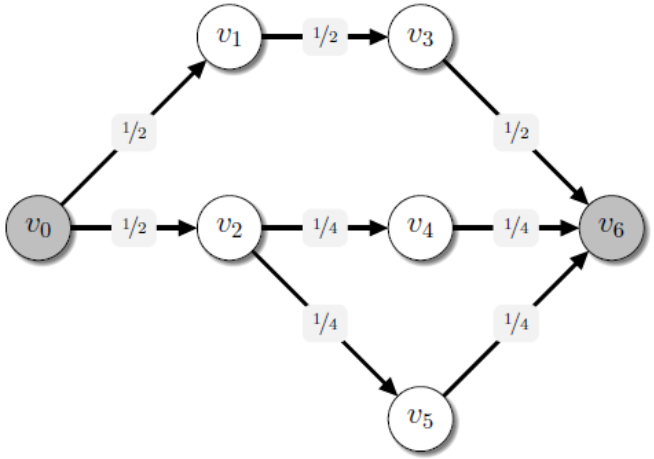
$$\sum_{i \in V : (i,v) \in S_t} x_{iv}^{d,t} \leq 1, \quad \forall d \in D\ ,\ \forall t \in T \text{ and } \forall v \in V \setminus \{s_d, t_d\},$$

MIP Model – Offline computation

- **r(i,j,a,t) :** fraction of flow from I to j that goes through arc a at time t.
- **Computation of the split coefficient r(i,j,a,t) :**
  1. Build G_t by removing arcs in q(t).
  2. Run Dijkstra on both G_t and the reversed G_t to get all shortest-path distance.
  3. Build FG(i,j) by keeping only the arcs which belong to any shortest path i.e. if

$$dist^*(i,j) = dist^*(i,u) + \omega(u,v) + dist^*(v,j)$$

  4. ECMP on vertices in FG(u,v) ➔ r(i,j,a,t).



Heuristic

Heuristic : Actual experimental setup

How each run is structured:

1. Offline compute split coefficients r(i,j,a,t)

2. H1 greedy demand insertion → feasible solution

3. Gurobi warm-started, lexicographic loop

All three phases share one **10-minute** wall-clock cap per instance.

Heuristic: Why warm-start Gurobi with a heuristic?

- ***Pruning :*** The MLU returned by H1 immediately becomes an upper bound on z. The Branch and bound tree is heavily pruned. Many nodes are cut before the root LP is even solved.

- ***Never worse than H1 :*** Even if the 10-min cap fires before optimality is proved, the incumbent returned by Gurobi is, by construction, at least as good as the heuristic solution.

- ***Lex loop reuse :*** At iteration ℓ, the solution from $\ell-1$ is feasible (only adds $\lambda \leq z^*$). We reuse it as warm start, so search effort accumulates instead of restarting.

Overall, any branch and bound effort already spent is preserved across the lex loop.

H1 : greedy demand insertion (MLU-driven)

- **Sort demands:** Decreasing total volume Σ_t v(d,t) — biggest demands placed first.
- **Enumerate paths:** For each (d,t): the direct (s,t) and every 1-waypoint path (s,w)+(w,t), with maxSeg ≤ 2.
- **Score and pick:** Choose the candidate giving the smallest post-insertion MLU; sum-of-loads as tiebreak (lex proxy).
- **Check budget k(t):** Discard candidates that exceed the reconfiguration budget vs. slot t−1 → solution feasible w.r.t.

Heuristic : what makes H1 scale

- **MLU and Σloads tracked incrementally:** no full recomputation between candidates.

- **Only the arcs touched by a candidate:** are scanned, not the whole |A|.

- **Scales to instances with thousands of demands**

(e.g. inst 16: |D|=4800, H1 < 13 s).

This scaling allows us to find a "good" first solution very fast to prune de branch and bound.

Heuristic : H1 warmstart limitations

- **MIP scaling wall:** Warm start doesn't change the MIP size
- **One-shot heuristic, no improvement loop:** H1 places each demand once and never reconsider
- The warm start is only locally consistent with the lex objective

Heuristic : what could have been done

- **A local search / metaheuristic on top of H1.** Take H1's solution and try local moves: swap two demands' paths, re-route a single demand through a different waypoint, accept the move if the lex vector improves.

- **A second heuristic H2 with a different bias**: then take the better of H1 / H2 as the warm start.

- **LP relaxation rounding** to generate a second warm start independent of H1, gives Gurobi two incumbents to choose from.

(those were not implemented due to lack of time)

Column restriction methods

Motivations and main idea

- Too many variables $x_{ij}^{d,t}$ $(|D| \times |S_t| \times |T|)$
- Model construction alone infeasible within the 10-minute time cap
- **Solution** : use a candidate set much smaller than the deliverable set

$$C_{d,t} \subseteq S_t$$

$$|C_{d,t}| \leq K \text{ for a parameter } K \ll |S_t|$$

- The candidate set should contain the segments that appear in the true optimal solution

1 - MLU scoring

- Computing a score for each demand *d* and for each deliverable segment *(i, j)*

$$score(d, t, i, j) = \max_{a \in A} \frac{f_{H1}(a, t) + r(i, j, a, t) v_{d,t}}{c_a}$$

- It estimates the load impact of routing demand *d* through segment *(i, j)* on top of H1 (estimates the resulting MLU)
- A low score means the segment routes demand *d* without heavily loading any arc
- It takes into account the fact that all other demands are already loading the network, so it selects segments that are good even on a loaded network

2 - Reachability filter

- For a given demand *(sd, td)*, a segment *(i, j)* is retained only if node *i* is reachable from *sd* in at most *maxSeg* – 1 jumps and node *j* can reach *td* in at most *maxSeg* – 1 jumps

$$C_{d,t} = \{(i,j) \in S_t : i \in \text{forward}(s_d) \cup \{s_d\} \text{ and } j \in \text{backward}(t_d) \ \cup \ \{t_d\}\}$$

- The forward and backward sets are computed according to the *Breadth-First Search* procedure
- To obtain *K* segments, the candidate set is arbitrarily truncated to the *K* first elements

Note

The following segments are always included into the candidate set :

- The direct path (*sd*, *td*) (simplest valid routing)
- The segments used by the H1 warm start (so that the final solution can never be worse than H1)

Method selection

- Selection based on the instance size

$$n_{ops} \approx |D| \times |S_t| \times |T|$$

Condition	K	Method
$n_{ops} \leq 50,000$	*none*	No restriction
$50,000 < n_{ops} \leq 100 \times 10^6$	50	MLU Scoring
$n_{ops} > 100 \times 10^6$	20	Reachability filter

Trade-off

- Trade-off between model size and solution quality
- A smaller *K* gives less routing alternatives and leads to less accurate results but decreases model construction time

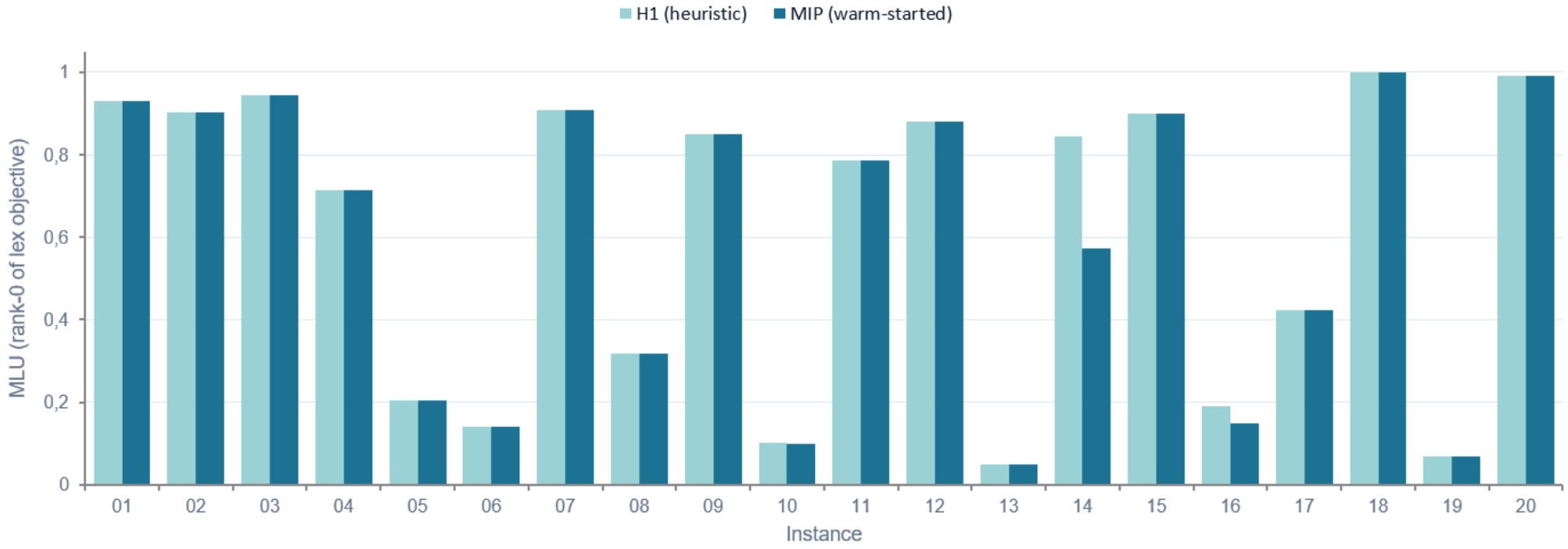
- **MLU scoring :** *K* = 50 (medium instances)

- **Reachability filter :** *K* = 20 (large instances)

Smaller *K* value for large instances to ensure feasibility over the 10-minute cap

Results

MLU per instance : H1 vs warm-started MIP



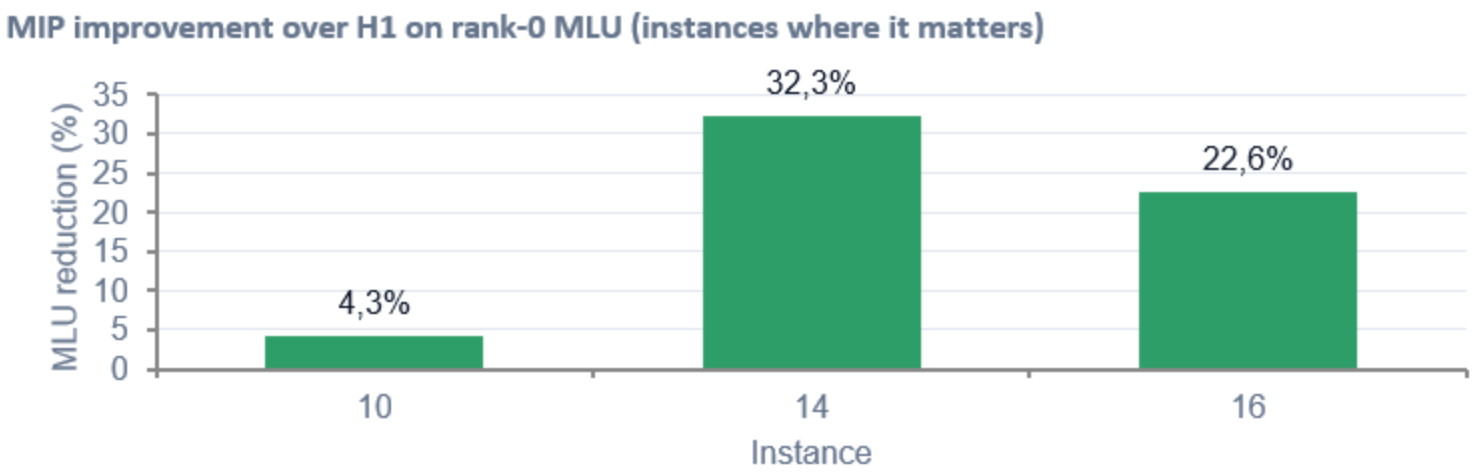
14 /18 H1 already optimal on rank-0

3 /18 MIP improved rank-0 MLU

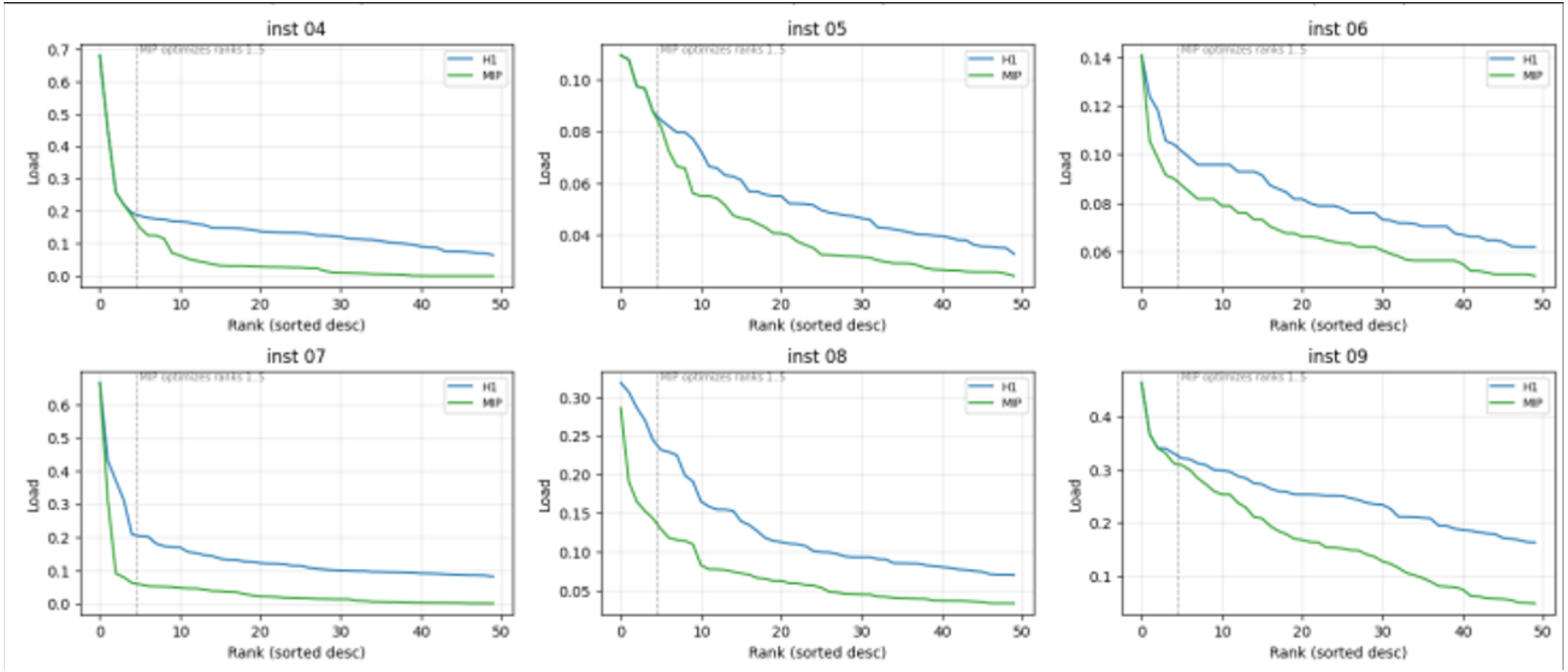
2 /20 no incumbent (inst 19, 20)

Where does the MIP actually pay off?

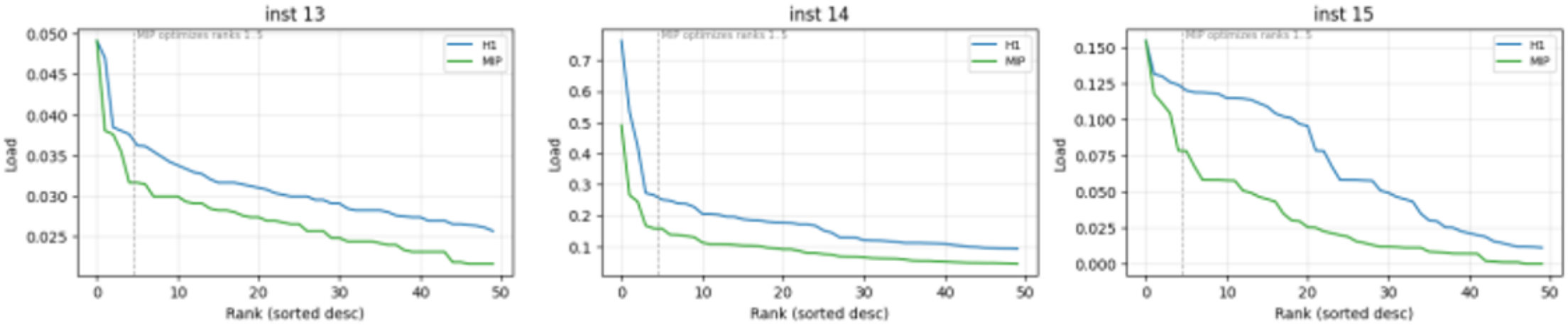
- Finding: H1 is already very good for rank-0. On 14/18 solves instances, H1 reaches optimality.



Results comparison (1): H1 vs gurobi warmstarted



Results comparison (2): H1 vs gurobi warmstarted



Limitations : for the result analysis we only optimized the first 5 lex. This could be easily extended to higher values.

We can see the dashed-line showing about to where our loads were fully optimized (lex_iter = 5). We can also see that our MIP stays better than our greedy heuristic for all values up to 50. It is because ranks 6-50 inherit benefits from the optimisation of ranks 1-5. Meaning the usage of the MIP of sheerly H1 was significant.

Limitations

MLU scoring (first limitation)

- It measures the impact of a segment on the most loaded arc, so it is rewards segments that improve the MLU (rank-0)
- But can reject segments that would help to optimize rank-1, rank-2, ...

Possible improvement :

$$score_\alpha(d,t,i,j) = (1-\alpha)\max_{a\in A}\frac{f_{H1}(a,t)+r(i,j,a,t)v_{d,t}}{c_a} + \alpha\sum_{a\in A}\frac{f_{H1}(a,t)+r(i,j,a,t)v_{d,t}}{c_a}$$

MLU scoring (second limitation)

- Parameter *K* is a global parameter applied for all demands
- But some demands may require different number of candidate K, depending on the connectivity between the source and the target
- It results in either unnecessary calculations or a lack of accuracy

Any questions ?